

Guía de Laboratorio 6

Entrega Grupal

Objetivo

- El objetivo de este laboratorio es avanzar en su proyecto y resolver dudas.
- Trabajarán sobre su dominio propio.
- No es necesario que el sistema esté completo hoy, pero sí que compile y ejecute sin errores antes de entregar.
- Etiqueta de requisito: al completar cada tarea, agreguen un comentario `// [REQUISITO: X]` en el código. Facilita la revisión.
- Recuerde documentar todos sus supuestos de diseño y el porqué de su elección.

Tareas (10 pts c/u)

Tarea 0 · Packages — organizar el proyecto (10 min)

Antes de escribir código nuevo, organicen el proyecto en paquetes.

1. En NetBeans, creen al menos dos paquetes según su dominio. Ejemplos:
 - a. modelo (clases de negocio: Producto, Cliente, etc.)
 - b. sistema (clase gestora y main)
2. Muevan cada clase al paquete correspondiente arrastrándola en el panel Projects. NetBeans actualiza los imports automáticamente.
3. Ejecuten el proyecto. Debe compilar sin errores antes de continuar.

Etiqueten en los comentarios de cada clase: `// [REQUISITO: Package]`

Tarea 1 · Clase abstracta y subclases (30 min)

1. Identifiquen la clase central de su sistema y conviértanla en abstract.
2. Agreguen al menos un método abstracto con sentido en su dominio.
3. Creen dos subclases concretas que lo implementen con lógica propia.

Checkpoint: En main, instancien un objeto de cada subclase usando el tipo de la clase abstracta. Llamen al método abstracto y verifiquen que cada subclase responde diferente. Ejecuten.

Si funciona: `// [REQUISITO: Herencia]`

Tarea 2 · Interface (20 min)

1. Definan una interface relevante para su dominio:
 - a. Ventas → interface `Vendible { double calcularTotal(); }`
 - b. Biblioteca → interface `Prestable { boolean estaDisponible(); }`
 - c. Gimnasio → interface `Evaluable { String obtenerNivel(); }`
2. Hagan que al menos una subclase la implemente (también la podría implementar la superclase).

Al terminar: compilen y verifiquen que la clase que implementa la interface no tiene errores.

Etiqueten: `// [REQUISITO: Interface]`

Tarea 3 · Enum (15 min)

1. Identifiquen un concepto con valores fijos (estado, categoría, tipo).
2. Créenlo como enum y úsenlo en alguna clase.

Al terminar: compilen. Etiqueten: `// [REQUISITO: Enum]`

Tarea 4 · Colecciones y menú con Scanner (25 min)

En la clase gestora del menú (no en main), declaren la estructura que almacenará los elementos principales de su sistema. Elijan la que más sentido tenga para su dominio:

- ArrayList — si acceden a los elementos por posición o simplemente los recorren.
- HashMap — si necesitan buscar por un identificador único (nombre, código, RUT, etc.).

El menú debe permitir al menos: agregar un elemento a la estructura, listar todos y salir.

Al terminar: el menú debe ejecutarse sin errores con al menos 2 objetos cargados.

Etiqueten: `// [REQUISITO: Colecciones]` y `// [REQUISITO: Menu-Scanner]`

Tarea 5 · Persistencia con Serializable (20 min)

Guarden y recuperen el estado de su sistema usando serialización de objetos.

1. Implementen Serializable en todas las clases de modelo (la clase abstracta y sus subclasses).
2. En la clase gestora, implementen dos métodos: guardar() y cargar().
3. Agreguen al menú las opciones Guardar y Cargar.
4. Prueben cerrando y volviendo a ejecutar el programa — los datos deben persistir.

Al terminar: cierren y vuelvan a ejecutar el proyecto. Los datos deben aparecer al cargar.

Etiqueten: `// [REQUISITO: Persistencia]`

Entregables del laboratorio

- Código fuente del proyecto en NetBeans con los avances del día, comentado con las etiquetas `// [REQUISITO: X]`.
- Informe con sus supuestos de diseño, explicando qué hace cada una de sus clases, cuál es la función de las interfaces que diseñó y el porqué de la decisión de usar clases abstractas.
- Diagrama de clases actualizado según lo implementado hoy (puede ser a mano o digital).

Importante: Solo un integrante debe entregar el laboratorio.

- Formato nombre archivo: "matricula1_matricula2_matricula3.zip"